
REQUIREMENTS NOT MET

None

PROBLEMS ENCOUNTERED

None Encountered

FUTURE WORK/APPLICATIONS

This lab teaches us the usage of Serial Peripheral Interface (SPI), which is fundamental to high-speed transmissions, by connecting a ‘master’ to a ‘student’ devices, we can have full-duplex communication from one to another, allowing us to have much faster rates than USART by sharing a clock signal (SCK). SPI also allows us to interface multiple devices or ‘students,’ by use of CS/SS. This could be used for high-speed sensors such as sound sensors, brightness sensors, etc. SPI is the ‘king of speed.’

PRE-LAB EXERCISES

- i. In regard to SPI communication that is to exist between the relevant ATxmega128A1U and IMU chips, answer each of the questions within the previously given bulleted list.
 - a. Which device(s) should be given the role of master and which device(s) should be given the role of student?
 - i. The microcontroller should be given the master role to control the clock signal (SCK), chip selects, and the initialization of all read/write transmissions.
 - ii. The IMU chip should be given the student role to be driven by the microcontroller, remaining inactive except for when the master enables it via chip select and drives it with SCK, allowing it to transmit and be read from.
 - b. How will the student device(s) be enabled? If a student select is utilized, rather than just have the device(s) be permanently enabled, which pin(s) will be used?
 - i. The IMU will be enabled via an active-low CS/SS signal, which is driven by pin 12 for both chips.
 - c. What is the order of data transmission? Is the MSb or LSb transmitted first?
 - i. MSB in/out first down to LSB in/out last per IMU specification.
 - d. In regard to the relevant clock signal, should data be latched on a rising edge or on a falling edge?
 - i. It should be latched on the rising edge, per IMU timing diagram (page 25), with new data bits transmitted on the falling edge.
 - e. What is the maximum serial clock frequency that can be utilized by the relevant devices?
 - i. The IMU Timing Values chart shows a maximum SPI clock frequency of 10MHz.
- ii. Why is it a better idea to modify global flag variables inside of ISRs instead of doing everything inside of them?
 - a. To reduce the time spent in the ISR, allowing the program to run the main loop as often as possible, and allowing other interrupts to fire as necessary (otherwise they are blocked.) Utilizing communication protocols within the ISR can also lead to race conditions.
- iii. To output two unsigned 32-bit values 0x42676937 [CH1] and 0xDEADBEEF [CH2] to SerialPlot, list all the bytes in the order you would send them via UART.
 - a. Little endian, from right to left: 0x37, 0x69, 0x67, 0x42; 0xEF, 0xBE, 0xAD, 0xDE.
- iv. What is the most positive value that can be received from the accelerometer (in decimal)? What about the most negative?
 - a. If bit 16 (2^{15}) determines sign, we can say that we are able to say that we can show $2^{15} - 1$, or 32,767 down to -2^{15} , or -32,768.

PSEUDOCODE/FLOWCHARTS

spi.c

```
/*-----  
spi.c --  
  
Description:  
    Provides useful definitions for manipulating the relevant SPI  
    module of the ATxmega128A1U.  
  
Author(s): Dr. Eric M. Schwartz, Christopher Crary, Wesley Piard  
Last modified by: Dr. Eric M. Schwartz  
Last modified on: 30 May 2026  
-----*/  
  
/*****DEPENDENCIES*****/  
  
#include <avr/io.h>  
#include "spi.h"  
  
/*****END OF DEPENDENCIES*****/  
  
/*****FUNCTION DEFINITIONS*****/  
  
void spi_init(void)  
{  
  
    /* Initialize the relevant SPI output signals to be in an "idle" state.  
     * Refer to the relevant timing diagram within the LSM6DSL datasheet.  
     * (You may wish to utilize the macros defined in `spi.h`.) */  
    PORTF.OUTSET = SS | SCK;  
  
    /* Configure the pin direction of relevant SPI signals. */  
    PORTF.DIRSET = MOSI | SS | SCK;  
    PORTF.DIRCLR = MISO;  
  
    /* Set the other relevant SPI configurations. */  
    SPIF.CTRL = SPI_PRESCALER_DIVX_gc |  
                SPI_MASTER_bm |  
                SPI_MODE_3_gc |  
                SPI_ENABLE_bm;  
}  
  
void spi_write(uint8_t data)  
{  
    /* Write to the relevant DATA register. */  
    SPIF.DATA = data;  
  
    /* Wait for relevant transfer to complete. */  
    while(IF FLAG NOT TRUE);  
  
    /* In general, it is probably wise to ensure that the relevant flag is  
     * cleared at this point, but, for our contexts, this will occur the  
     * next time we call the `spi_write` (or `spi_read`) routine.  
     * Really, because of how the flag must be cleared within  
     * ATxmega128A1U, it would probably make more sense to have some single  
     * function, say `spi_transceive`, that both writes and reads
```

```
    * data, rather than have two functions `spi_write` and `spi_read`,  
    * but we will not concern ourselves with this possibility  
    * during this semester of the course. */  
}  
  
uint8_t spi_read(void)  
{  
    /* Write some arbitrary data to initiate a transfer. */  
    SPIF.DATA = 0x37;  
  
    /* Wait for relevant transfer to be complete. */  
    while(IF FLAG NOT TRUE);  
  
    /* After the transmission, return the data that was received. */  
    return SPIF.DATA;  
}  
  
/*****END OF FUNCTION DEFINITIONS*****/
```

Lab6_2.c

```
/*
 * lab6_2.c
 *
 * Created: 7/12/2026 2:52:31 PM
 * Author : Evan Baesler
 */

#include <avr/io.h>
#include "spi.h"

int main(void)
{
    // Initialize the SPI module
    call spi_init

    while(1)
    {
        PORTF.OUTCLR = SS; // Simulate turning IMU CS on

        // Transmit 0x42
        spi_write(0x42)

        PORTF.OUTSET = SS; // Simulate turning IMU CS off
    }
}

/*****FUNCTION DEFINITIONS*****/

void spi_init(void)
{
    /* Initialize the relevant SPI output signals to be in an "idle" state.
     * Refer to the relevant timing diagram within the LSM6DSL datasheet.
     * (You may wish to utilize the macros defined in `spi.h`.) */
    PORTF.OUTSET = SS | SCK;

    /* Configure the pin direction of relevant SPI signals. */
    PORTF.DIRSET = MOSI | SS | SCK;
    PORTF.DIRCLR = MISO;

    /* Set the other relevant SPI configurations. */
    SPIF.CTRL = SPI_PRESCALER_DIVX_gc |
                SPI_MASTER_bm |
                SPI_MODE_3_gc |
                SPI_ENABLE_bm;
}

void spi_write(uint8_t data)
{
    /* Write to the relevant DATA register. */
    SPIF.DATA = data;

    /* Wait for relevant transfer to complete. */
    while(IF FLAG NOT TRUE);

    /* In general, it is probably wise to ensure that the relevant flag is
     * cleared at this point, but, for our contexts, this will occur the
     * next time we call the `spi_write` (or `spi_read`) routine.
     */
}
```

```
    * Really, because of how the flag must be cleared within
    * ATxmega128A1U, it would probably make more sense to have some single
    * function, say `spi_transceive`, that both writes and reads
    * data, rather than have two functions `spi_write` and `spi_read`,
    * but we will not concern ourselves with this possibility
    * during this semester of the course. */
}

uint8_t spi_read(void)
{
    /* Write some arbitrary data to initiate a transfer. */
    SPIF.DATA = 0x37;

    /* Wait for relevant transfer to be complete. */
    while(IF FLAG NOT TRUE);

    /* After the transmission, return the data that was received. */
    return SPIF.DATA;
}

/*****END OF FUNCTION DEFINITIONS*****/
```

Lab6_3.c

Include dependencies (spi, lsm6dsl, io)

Write Prototypes

Main

{

spi_init

uint8_t output = 0

while(true)

{

output = LSM_read(WHO_AM_I)

}

}

// SUBROUTINES:

uint8_t LSM_read(reg_addr) // uint8_t to read the value of a register

{

Enable IMU

Read from address (MSB ON)

WAIT FOR TRANSFER

Read a dummy value

WAIT FOR TRANSFER

Disable IMU

Return Data from Address

}

void LSM_write(reg_addr, data)

{

Enable IMU

Write to address (MSB OFF)

WAIT FOR TRANSFER

Write value

WAIT FOR TRASNFER

Disable IMU

}

Reuse SPI_init from prior section

Lab6_4

Include dependencies (spi, lsm6dsl, io)

Write Prototypes

Main

{

spi_init

LSM_init

sei

while(1)

{

// INFINITE WAIT LOOP

}

}

// SUBROUTINES

Reuse spi_init, LSM_read/write

void LSM_init(void)

{

Set SW_Reset flag

Delay

Set interrupt to active-low

Enable data-ready interrupt

Disable DEN in output

Set up sampling frequency and sensitivity

Set INT1 to relevant pin

Set low level interrupt sense

Enable low level interrupts

}

Void usartd0_out_repeat(*data, length)

{

For (I = 0; I < length; i++)

{

Usartd0_out_char(data[i])a // PROVIDED

}

}

// INTERRUPT

Interrupt(PORTC_INT0)

{

Output = LSM_READ(OUTX_L_XL)

Reset interrupt flag

}

Lab6_5

Include dependencies (io, interrupt, spi, uart, lsm)

Include prior subroutines (LSM functions, spi, usart, repeat)

Volatile accel_flag

Accel_data[6] = {0,0,0,0,0,0}

Main

{

Spi_init

LSM_init

Usartd0_init

While(1)

{

If accel_flag

{

Accel_data[0-5] gets X-Z data

Usartd0_out_repeat(accel_data, 6)

Reset accel flag

Enable INT0 on PORTC

}

}

}

// INTERRUPT

Interrupt(PORTC_INT0)

{

Accel_flag = 1

Disable interrupt on PORTC

Reset interrupt flag

}

Lab6_6

Include dependencies (io, interrupt, spi, uart, lsm)

Include prior subroutines (LSM read/write functions, spi, usart, repeat)

Volatile accel_flag

gyro_data[6] = {0,0,0,0,0,0}

Main

{

Spi_init

LSM_init

Usartd0_init

While(1)

{

If gyro_flag

{

Accel_data[0-5] gets X-Z data

Usartd0_out_repeat(accel_data, 6)

Reset gyro flag

Enable INT0 on PORTC

}

}

}

// INTERRUPT

Interrupt(PORTC_INT0)

{

Gyro_flag = 1

Disable interrupt on PORTC

Reset interrupt flag

}

```
void LSM_init(void)
{
    Set SW_Reset flag
    Delay
    Set interrupt to active-low
    Enable data-ready interrupt for gyro
    Disable DEN in output
    Set up sampling frequency and sensitivity for gyro

    Set INT1 to relevant pin
    Set low level interrupt sense
    Enable low level interrupts
}
```

PROGRAM CODE

```
spi.h
#ifndef SPI_H_                // Header guard.
#define SPI_H_

/*-----
spi.h --

Description:
    Provides function prototypes and macro definitions for utilizing the SPI
    system of the ATxmega128A1U.

Author(s): Dr. Eric M. Schwartz, Christopher Crary, Wesley Piard
Last modified by: Dr. Eric M. Schwartz
Last modified on: 30 May 2026
Last configured by: Evan Baesler
Last configured on: 12 July 2026
-----*/

/*****DEPENDENCIES*****/

#include <avr/io.h>

/*****END OF DEPENDENCIES*****/

/*****MACROS*****/

#define SS_bm    (1<<4)
#define MOSI_bm  (1<<5)
#define MISO_bm  (1<<6)
#define SCK_bm   (1<<7) // Per PORTF Alternate Functions

/*****END OF MACROS*****/

/*****FUNCTION PROTOTYPES*****/

/*-----
spi_init --

Description:
    Initializes the relevant SPI module to communicate with the LSM6DSL.

Input(s): N/A
Output(s): N/A
-----*/
void spi_init(void);

/*-----
spi_write --

Description:
    Transmits a single byte of data via the relevant SPI module.

Input(s): `data` - 8-bit value to be written via the relevant SPI module.
Output(s): N/A
-----*/
void spi_write(uint8_t data);

/*-----
```

`spi_read --`

Description:

Reads a byte of data via the relevant SPI module.

Input(s): N/A

Output(s): 8-bit value read from the relevant SPI module.

```
-----*/  
uint8_t spi_read(void);  
  
/*****END OF FUNCTION PROTOTYPES*****/  
  
#endif // End of header guard.
```

spi.c

```
/*-----  
spi.c --  
  
Description:  
    Provides useful definitions for manipulating the relevant SPI  
    module of the ATxmega128A1U.  
  
Author(s): Dr. Eric M. Schwartz, Christopher Crary, Wesley Piard  
Last modified by: Dr. Eric M. Schwartz  
Last modified on: 30 May 2026  
Last configured by: Evan Baesler  
Last configured on: 12 July 2026  
-----*/  
  
/*****DEPENDENCIES*****/  
  
#include <avr/io.h>  
#include "spi.h"  
  
/*****END OF DEPENDENCIES*****/  
  
/*****FUNCTION DEFINITIONS*****/  
  
void spi_init(void)  
{  
  
    /* Initialize the relevant SPI output signals to be in an "idle" state.  
     * Refer to the relevant timing diagram within the LSM6DSL datasheet.  
     * (You may wish to utilize the macros defined in `spi.h`.) */  
    PORTF.OUTSET = SS_bm | SCK_bm;  
  
    /* Configure the pin direction of relevant SPI signals. */  
    PORTF.DIRSET = SS_bm | MOSI_bm | SCK_bm;  
    PORTF.DIRCLR = MISO_bm;  
  
    /* Set the other relevant SPI configurations. */  
    SPIF.CTRL = SPI_PRESCALER_DIV16_gc |  
                SPI_MASTER_bm |  
                SPI_MODE_3_gc |  
                SPI_ENABLE_bm;  
    // This leaves CTRL.DORD false, so MSB -> LSB  
  
}  
  
void spi_write(uint8_t data)  
{  
    /* Write to the relevant DATA register. */  
    SPIF.DATA = data;  
  
    /* Wait for relevant transfer to complete. */  
    while(!(SPIF.STATUS & (SPI_IF_bm)));  
  
    /* In general, it is probably wise to ensure that the relevant flag is  
     * cleared at this point, but, for our contexts, this will occur the  
     * next time we call the `spi_write` (or `spi_read`) routine.  
     * Really, because of how the flag must be cleared within  
     * ATxmega128A1U, it would probably make more sense to have some single  
     * function, say `spi_transceive`, that both writes and reads  
     * data, rather than have two functions `spi_write` and `spi_read`,
```

```
    * but we will not concern ourselves with this possibility
    * during this semester of the course. */
}

uint8_t spi_read(void)
{
    /* Write some arbitrary data to initiate a transfer. */
    SPIF.DATA = 0x37;

    /* Wait for relevant transfer to be complete. */
    while(!(SPIF.STATUS & (SPI_IF_bm)));

    /* After the transmission, return the data that was received. */
    return SPIF.DATA;
}

/*****END OF FUNCTION DEFINITIONS*****/
```

```
USART.H
// Header guard.
#define USART_H

/*-----
USART.H --

Description:
    Provides some useful declarations regarding the USART system of the
    ATxmega128A1U.

Author(s): Dr. Eric Schwartz, Christopher Crary, Wesley Piard
Last modified by: Dr. Eric M. Schwartz
Last modified on: 30 May 2026
-----*/

/*****DEPENDENCIES*****/

#include <avr/io.h>

/*****END OF DEPENDENCIES*****/

/*****MACROS*****/
/*****END OF MACROS*****/

/*****CUSTOM DATA TYPES*****/
/*****END OF CUSTOM DATA TYPES*****/

/*****FUNCTION PROTOTYPES*****/

/*-----
USARTD0_IN_CHAR --

Description:
    Returns a single character via the receiver of the USARTD0 module.

Input(s): N/A
Output(s): Character received from USARTD0 module.
-----*/
char USARTD0_IN_CHAR(void);

/*-----
USARTD0_IN_STRING --

Description:
    Reads in a string with the receiver of the USARTD0 module.

    The string is to be stored within a pre-allocated buffer, accessible
    via the character pointer `buf`.

Input(s): `buf` - Pointer to character buffer.
Output(s): N/A
-----*/
void USARTD0_IN_STRING(char * buf);

/*-----
USARTD0_INIT --

Description:
    Configures the USARTD0 module for a specific asynchronous serial protocol.
```

```
    Input(s): N/A
    Output(s): N/A
-----*/
void usartd0_init(void);

/*-----
    usartd0_out_char --

    Description:
        Outputs a character via the transmitter of the USARTD0 module.

    Input(s): `c` - Read-only character.
    Output(s): N/A
-----*/
void usartd0_out_char(char c);

/*-----
    usartd0_out_string --

    Description:
        Outputs a string via the transmitter of the USARTD0 module.

        The string is to be stored within a pre-allocated buffer, accessible
        via the character pointer `str`.

    Input(s): `str` - Pointer to read-only character string.
    Output(s): N/A
-----*/
void usartd0_out_string(const char * str);

/*****END OF FUNCTION PROTOTYPES*****/

#endif          // End of header guard.
```

usart.c

```
/*-----  
usart.c --  
  
Description:  
    Provides some useful definitions regarding the USART system of the  
    ATxmega128A1U.  
  
Author(s): Dr. Eric Schwartz, Christopher Crary, Wesley Piard  
Last modified by: Dr. Eric M. Schwartz  
Last modified on: 30 May 2026  
-----*/  
  
/*****DEPENDENCIES*****/  
  
#include <avr/io.h>  
#include "usart.h"  
  
/*****END OF DEPENDENCIES*****/  
  
/*****MACROS*****/  
  
/* At 2 MHz SYSclk, 5 BSEL, -6 BSCALE corresponds to 115200 bps */  
#define BSEL    (5)  
#define BSCALE  (-6)  
  
/*****END OF MACROS*****/  
  
/*****FUNCTION DEFINITIONS*****/  
  
char usartd0_in_char(void)  
{  
    /* intentionally left blank */  
}  
  
void usartd0_in_string(char * buf)  
{  
    /* intentionally left blank */  
}  
  
void usartd0_init(void)  
{  
    /* Configure relevant TxD and RxD pins. */  
    PORTD.OUTSET = PIN3_bm;  
    PORTD.DIRSET = PIN3_bm;  
    PORTD.DIRCLR = PIN2_bm;  
  
    /* Configure baud rate. */  
    USARTD0.BAUDCTRLA = (uint8_t)BSEL;  
    USARTD0.BAUDCTRLB = (uint8_t)((BSCALE << 4)|(BSEL >> 8));  
  
    /* Configure remainder of serial protocol. */  
    /* (In this example, a protocol with 8 data bits, no parity, and  
    * one stop bit is chosen.) */  
    USARTD0.CTRLC = (USART_CMODE_ASYNCHRONOUS_gc |  
                     USART_PMODE_DISABLED_gc    |  
                     USART_CHSIZE_8BIT_gc)       &  
                     ~USART_SBMODE_bm;  
  
    /* Enable receiver and/or transmitter systems. */
```

```
    USARTD0.CTRLB = USART_RXEN_bm | USART_TXEN_bm;

    /* Enable interrupt (optional). */
    /* USARTD0.CTRLA = USART_RXCINTLVL_MED_gc; */
}

void usartd0_out_char(char c)
{
    while(!(USARTD0.STATUS & USART_DREIF_bm));
    USARTD0.DATA = c;
}

void usartd0_out_string(const char * str)
{
    while(*str) usartd0_out_char(*(str++));
}

/*****END OF FUNCTION DEFINITIONS*****/
```

lsm6dsl.h

```
#ifndef LSM6DSL_H_ // Header guard.
#define LSM6DSL_H_

/*-----
lsm6dsl.h --

Description:
  Provides custom data types to make it easier to handle any data
  read from the LSM6DSL IMU.

  The LSM6DSL can output accelerometer and gyroscope data. Data from both
  of these sensors is represented in a 16-bit signed format.

  Author(s): Wesley Piard & Dr. Eric M. Schwartz
  Last modified by: Dr. Eric M. Schwartz
  Last modified on: 30 May 2026
-----*/

/*****MACROS*****/

#define LSM6DSL_SPI_READ_STROBE_bm      0x80
#define LSM6DSL_SPI_WRITE_STROBE_bm    0x00

/*****END OF MACROS*****/

/*****CUSTOM DATA TYPES*****/

/* Used to differentiate the accelerometer and gyroscope within the LSM6DSL. */
typedef enum {LSM6DSL_ACCEL, LSM6DSL_GYRO} lsm6dsl_module_t;

/* Can be used to contain the separated bytes of data as they are read from
 * the LSM6DSL. */
typedef struct lsm6dsl_data_raw
{
    uint8_t accel_x_low, accel_x_high;
    uint8_t accel_y_low, accel_y_high;
    uint8_t accel_z_low, accel_z_high;

    uint8_t gyro_x_low, gyro_x_high;
    uint8_t gyro_y_low, gyro_y_high;
    uint8_t gyro_z_low, gyro_z_high;
}lsm6dsl_data_raw_t;

/* Contains the full concatenated signed 16-bit words of data. */
typedef struct lsm6dsl_data_full
{
    int16_t accel_x, accel_y, accel_z;
    int16_t gyro_x, gyro_y, gyro_z;
}lsm6dsl_data_full_t;

/* Provides the ability to choose how to access the LSM6DSL data. */
typedef union lsm6dsl_data
{
    lsm6dsl_data_full_t word;
    lsm6dsl_data_raw_t byte;
}lsm6dsl_data_t;
```

```
/******END OF CUSTOM DATA TYPES******/
```

```
/******FUNCTION PROTOTYPES******/
```

```
uint8_t LSM_read(uint8_t reg_addr)
{
    PORTF.OUTCLR = SS_bm; // Enable IMU

    SPIF.DATA = (reg_addr | 0x80); // READ FROM ADDRESS (MSB ON)

    while(!(SPIF.STATUS & (SPI_IF_bm))); // WAIT FOR TRANSFER

    SPIF.DATA = 0x37; // DUMMY VALUE, OVERWRITTEN IN NEXT CLK

    while(!(SPIF.STATUS & (SPI_IF_bm))); // WAIT FOR TRANSFER

    PORTF.OUTSET = SS_bm; // Disable IMU

    return SPIF.DATA; // RETURN DATA FROM ADDRESS
}
```

```
void LSM_write(uint8_t reg_addr, uint8_t data)
{
    PORTF.OUTCLR = SS_bm; // Enable IMU

    SPIF.DATA = (reg_addr); // WRITE TO ADDRESS (MSB OFF)

    while(!(SPIF.STATUS & (SPI_IF_bm))); // WAIT FOR TRANSFER

    SPIF.DATA = data; // WRITTEN VALUE

    while(!(SPIF.STATUS & (SPI_IF_bm))); // WAIT FOR TRANSFER

    PORTF.OUTSET = SS_bm; // Disable IMU
}
```

```
/******END OF FUNCTION PROTOTYPES******/
```

```
#endif // End of header guard.
```

lsm6dsl_registers.h

```
#ifndef LSM6DSL_REGISTERS_H_           // Header guard.

#define LSM6DSL_REGISTERS_H_

/*-----
lsm6dsl_registers.h --

Description:
  Provides useful macro definitions and symbols that can be used
  when accessing registers of the LSM6DSL IMU.

Created by: 00TB-LT
Created on: 14 September 2019

Last modified by: Dr. Eric M. Schwartz
Last modified on: 30 May 2026
-----*/

/*****MACROS*****/
/*****END OF MACROS*****/

/*****CUSTOM DATA TYPES*****/

typedef enum LSM6DSL_ACCEL_REGISTERS
{
    FUNC_CFG_ACCESS                = 0x01,
    SENSOR_SYNC_TIME_FRAME        = 0x04,
    SENSOR_SYNC_RES_RATIO         = 0x05,

    FIFO_CTRL1                    = 0x06,
    FIFO_CTRL2                    = 0x07,
    FIFO_CTRL3                    = 0x08,
    FIFO_CTRL4                    = 0x09,
    FIFO_CTRL5                    = 0x0A,

    DRDY_PULSE_CFG_G              = 0x0B,

    INT1_CTRL                     = 0x0D,
    INT2_CTRL                     = 0x0E,

    WHO_AM_I                      = 0x0F,

    CTRL1_XL                      = 0x10,

    CTRL2_G                       = 0x11,

    CTRL3_C                       = 0x12,
    CTRL4_C                       = 0x13,
    CTRL5_C                       = 0x14,
    CTRL6_C                       = 0x15,

    CTRL7_G                       = 0x16,

    CTRL8_XL                      = 0x17,
    CTRL9_XL                      = 0x18,
    CTRL10_C                     = 0x19,
```

```

MASTER_CONFIG          =      0x1A,

WAKE_UP_SRC             =      0x1B,

TAP_SRC                 =      0x1C,
D6D_SRC                 =      0x1D,
STATUS_REG              =      0x1E,

OUT_TEMP_L              =      0x20,
OUT_TEMP_H              =      0x21,

OUTX_L_G                =      0x22,
OUTX_H_G                =      0x23,
OUTY_L_G                =      0x24,
OUTY_H_G                =      0x25,
OUTZ_L_G                =      0x26,
OUTZ_H_G                =      0x27,

OUTX_L_XL               =      0x28,
OUTX_H_XL               =      0x29,
OUTY_L_XL               =      0x2A,
OUTY_H_XL               =      0x2B,
OUTZ_L_XL               =      0x2C,
OUTZ_H_XL               =      0x2D,

SENSORHUB1_REG          =      0x2E,
SENSORHUB2_REG          =      0x2F,
SENSORHUB3_REG          =      0x30,
SENSORHUB4_REG          =      0x31,
SENSORHUB5_REG          =      0x32,
SENSORHUB6_REG          =      0x33,
SENSORHUB7_REG          =      0x34,
SENSORHUB8_REG          =      0x35,
SENSORHUB9_REG          =      0x36,
SENSORHUB10_REG         =      0x37,
SENSORHUB11_REG         =      0x38,
SENSORHUB12_REG         =      0x39,

FIFO_STATUS1            =      0x3A,
FIFO_STATUS2            =      0x3B,
FIFO_STATUS3            =      0x3C,
FIFO_STATUS4            =      0x3D,

FIFO_DATA_OUT_L         =      0x3E,
FIFO_DATA_OUT_H         =      0x3F,

TIMESTAMP0_REG          =      0x40,
TIMESTAMP1_REG          =      0x41,
TIMESTAMP2_REG          =      0x42,

STEP_TIMESTAMP_L        =      0x49,
STEP_TIMESTAMP_H        =      0x4A,

STEP_COUNTER_L          =      0x4B,
STEP_COUNTER_H          =      0x4C,

SENSORHUB13_REG         =      0x4D,
SENSORHUB14_REG         =      0x4E,
SENSORHUB15_REG         =      0x4F,
SENSORHUB16_REG         =      0x50,

```

```
    SENSORHUB17_REG          = 0x51,
    SENSORHUB18_REG          = 0x52,

    FUNC_SRC1                 = 0x53,
    FUNC_SRC2                 = 0x54,

    WRIST_TILT_IA             = 0x55,

    TAP_CFG                   = 0x58,
    TAP_THS_6D                = 0x59,

    INT_DUR2                  = 0x5A,

    WAKE_UP_THS               = 0x5B,
    WAKE_UP_DUR               = 0x5C,

    FREE_FALL                 = 0x5D,

    MD1_CFG                   = 0x5E,
    MD2_CFG                   = 0x5F,

    MASTER_CMD_CODE           = 0x60,

    SENS_SYNC_SPI_ERROR_CODE  = 0x61,

    OUT_MAG_RAW_X_L           = 0x66,
    OUT_MAG_RAW_X_H           = 0x67,
    OUT_MAG_RAW_Y_L           = 0x68,
    OUT_MAG_RAW_Y_H           = 0x69,
    OUT_MAG_RAW_Z_L           = 0x6A,
    OUT_MAG_RAW_Z_H           = 0x6B,

    X_OFS_USR                 = 0x73,
    Y_OFS_USR                 = 0x74,
    Z_OFS_USR                 = 0x75

}LSM6DSL_REGA_t;

/*****END OF CUSTOM DATA TYPES*****/

#endif // End of header guard.

Lab6_2

/*
 * lab6_2.c
 *
 * Created: 7/12/2026 2:52:31 PM
 * Author : Evan Baesler
 */

#include <avr/io.h>
#include "spi.h"

int main(void)
{
    // Initialize the SPI module
    spi_init();

    while(1)
    {
```

```
        PORTF.OUTCLR = SS_bm; // Simulate turning IMU CS on

        // Transmit 0x42
        spi_write(0x42);

        PORTF.OUTSET = SS_bm; // Simulate turning IMU CS off
    }
}

/*****FUNCTION DEFINITIONS*****/

void spi_init(void)
{
    /* Initialize the relevant SPI output signals to be in an "idle" state.
     * Refer to the relevant timing diagram within the LSM6DSL datasheet.
     * (You may wish to utilize the macros defined in `spi.h`.) */
    PORTF.OUTSET = SS_bm | SCK_bm;

    /* Configure the pin direction of relevant SPI signals. */
    PORTF.DIRSET = SS_bm | MOSI_bm | SCK_bm;
    PORTF.DIRCLR = MISO_bm;

    /* Set the other relevant SPI configurations. */
    SPIF.CTRL = SPI_PRESCALER_DIV16_gc |
                SPI_MASTER_bm |
                SPI_MODE_3_gc |
                SPI_ENABLE_bm;
    // This leaves CTRL.DORD false, so MSB -> LSB
}

void spi_write(uint8_t data)
{
    /* Write to the relevant DATA register. */
    SPIF.DATA = data;

    /* Wait for relevant transfer to complete. */
    while(!(SPIF.STATUS & (SPI_IF_bm)));

    /* In general, it is probably wise to ensure that the relevant flag is
     * cleared at this point, but, for our contexts, this will occur the
     * next time we call the `spi_write` (or `spi_read`) routine.
     * Really, because of how the flag must be cleared within
     * ATxmega128A1U, it would probably make more sense to have some single
     * function, say `spi_transceive`, that both writes and reads
     * data, rather than have two functions `spi_write` and `spi_read`,
     * but we will not concern ourselves with this possibility
     * during this semester of the course. */
}

uint8_t spi_read(void)
{
    /* Write some arbitrary data to initiate a transfer. */
    SPIF.DATA = 0x37;

    /* Wait for relevant transfer to be complete. */
    while(!(SPIF.STATUS & (SPI_IF_bm)));

    /* After the transmission, return the data that was received. */
    return SPIF.DATA;
}
```

}

/*****END OF FUNCTION DEFINITIONS*****/

Lab6_3

```
/*
 * lab6_3.c
 *
 * Created: 7/14/2026 12:52:10 AM
 * Author: Evan Baesler
 */

#include <avr/io.h>
#include "spi.h"
#include "lsm6dsl.h"
#include "lsm6dsl_registers.h"

uint8_t LSM_read(uint8_t reg_addr);
void LSM_write(uint8_t reg_addr, uint8_t data); // PROTOTYPES

int main(void)
{
    spi_init();
    uint8_t output;

    while(1)
    {
        output = LSM_read(WHO_AM_I);
    }
}

uint8_t LSM_read(uint8_t reg_addr)
{
    PORTF.OUTCLR = SS_bm; // Enable IMU

    SPIF.DATA = (reg_addr | 0x80); // READ FROM ADDRESS (MSB ON)

    while(!(SPIF.STATUS & (SPI_IF_bm))); // WAIT FOR TRANSFER

    SPIF.DATA = 0x37; // DUMMY VALUE, OVERWRITTEN IN NEXT CLK

    while(!(SPIF.STATUS & (SPI_IF_bm))); // WAIT FOR TRANSFER

    PORTF.OUTSET = SS_bm; // Disable IMU

    return SPIF.DATA; // RETURN DATA FROM ADDRESS
}

void LSM_write(uint8_t reg_addr, uint8_t data)
{
    PORTF.OUTCLR = SS_bm; // Enable IMU

    SPIF.DATA = (reg_addr); // WRITE TO ADDRESS (MSB OFF)

    while(!(SPIF.STATUS & (SPI_IF_bm))); // WAIT FOR TRANSFER

    SPIF.DATA = data; // WRITTEN VALUE

    while(!(SPIF.STATUS & (SPI_IF_bm))); // WAIT FOR TRANSFER

    PORTF.OUTSET = SS_bm; // Disable IMU
}
```

```
}

void spi_init(void)
{
    /* Initialize the relevant SPI output signals to be in an "idle" state.
     * Refer to the relevant timing diagram within the LSM6DSL datasheet.
     * (You may wish to utilize the macros defined in `spi.h`.) */
    PORTF.OUTSET = SS_bm | SCK_bm;

    /* Configure the pin direction of relevant SPI signals. */
    PORTF.DIRSET = SS_bm | MOSI_bm | SCK_bm;
    PORTF.DIRCLR = MISO_bm;

    /* Set the other relevant SPI configurations. */
    SPIF.CTRL = SPI_PRESCALER_DIV16_gc |
                SPI_MASTER_bm |
                SPI_MODE_3_gc |
                SPI_ENABLE_bm;
    // This leaves CTRL.DORD false, so MSB -> LSB
}
```

Lab6_4

```
/*
 * lab6_4.c
 *
 * Created: 7/14/2026 2:10:29 AM
 * Author: Evan Baesler
 */

#include <avr/io.h>
#include <avr/interrupt.h>
#include "spi.h"
#include "lsm6dsl.h"
#include "lsm6dsl_registers.h"

void spi_init(void);
void LSM_init(void);
uint8_t LSM_read(uint8_t reg_addr);
void LSM_write(uint8_t reg_addr, uint8_t data); // PROTOTYPES
uint8_t output;

int main(void)
{
    spi_init();
    LSM_init();

    sei();

    while(1)
    {

        // NOOP TIL INTERRUPT IS READY

    }
}

// SUBROUTINES

void spi_init(void)
{
    /* Initialize the relevant SPI output signals to be in an "idle" state.
     * Refer to the relevant timing diagram within the LSM6DSL datasheet.
     * (You may wish to utilize the macros defined in `spi.h`.) */
    PORTF.OUTSET = SS_bm | SCK_bm;

    /* Configure the pin direction of relevant SPI signals. */
    PORTF.DIRSET = SS_bm | MOSI_bm | SCK_bm;
    PORTF.DIRCLR = MISO_bm;

    /* Set the other relevant SPI configurations. */
    SPIF.CTRL = SPI_PRESCALER_DIV16_gc |
                SPI_MASTER_bm |
                SPI_MODE_3_gc |
                SPI_ENABLE_bm;
    // This leaves CTRL.DORD false, so MSB -> LSB
}

void LSM_init(void)
{
    LSM_write(CTRL3_C, 0x01); // SW_RESET FLAG, SELF CLEARS
```

```
    for(uint16_t i = 0; i < 2000; i++);

    LSM_write(CTRL3_C, (1<<5)); // SET INTERRUPT TO ACTIVE-LOW
    LSM_write(INT1_CTRL, 0x01); // ENABLE INT1_DRDY_XL

    LSM_write(CTRL9_XL, 0); // DISABLE DEN IN DATA OUTPUT FOR HIGHER RES
    LSM_write(CTRL1_XL, 0b01010000); // 208Hz, +-2g
    // THIS MEANS 208 SAMPLES / SECOND

    PORTC.DIRCLR = PIN6_bm; // INT1 PIN
    PORTC.PIN6CTRL = PORT_ISC_LEVEL_gc; // LOW LEVEL INTERRUPT SENSE
    PORTC.INTCTRL = PORT_INT0LVL_LO_gc; // SET INT0 TO LO LVL INT
    PORTC.INT0MASK = PIN6_bm; // CONNECT PIN6 TO PORTC INT0
    PMIC.CTRL = PMIC_LOLVLEN_bm; // ENABLE LOLVL INT
}

void usartd0_out_repeat(uint8_t *data, uint8_t length)
{
    for (uint8_t i = 0; i < length; i++)
    {
        usartd0_out_char(data[i]);
    }
}

uint8_t LSM_read(uint8_t reg_addr)
{
    PORTF.OUTCLR = SS_bm; // Enable IMU

    SPIF.DATA = (reg_addr | 0x80); // READ FROM ADDRESS (MSB ON)

    while(!(SPIF.STATUS & (SPI_IF_bm))); // WAIT FOR TRANSFER

    SPIF.DATA = 0x37; // DUMMY VALUE, OVERWRITTEN IN NEXT CLK

    while(!(SPIF.STATUS & (SPI_IF_bm))); // WAIT FOR TRANSFER

    PORTF.OUTSET = SS_bm; // Disable IMU

    return SPIF.DATA; // RETURN DATA FROM ADDRESS
}

void LSM_write(uint8_t reg_addr, uint8_t data)
{
    PORTF.OUTCLR = SS_bm; // Enable IMU

    SPIF.DATA = (reg_addr); // WRITE TO ADDRESS (MSB OFF)

    while(!(SPIF.STATUS & (SPI_IF_bm))); // WAIT FOR TRANSFER

    SPIF.DATA = data; // WRITTEN VALUE

    while(!(SPIF.STATUS & (SPI_IF_bm))); // WAIT FOR TRANSFER

    PORTF.OUTSET = SS_bm; // Disable IMU
}

// INTERRUPTS

ISR(PORTC_INT0_vect)
{
```

```
        output = LSM_read(OUTX_L_XL);

        PORTC.INTFLAGS = PIN6_bm;
}Lab6_5
/*
 * lab6_5.c
 *
 * Created: 7/14/2026 2:32:51 PM
 * Author: Evan Baesler
 */

#include <avr/io.h>
#include <avr/interrupt.h>
#include "spi.h"
#include "usart.h"
#include "lsm6dsl.h"
#include "lsm6dsl_registers.h"

void spi_init(void);
void LSM_init(void);
void usartd0_out_repeat(uint8_t *data, uint8_t length);

volatile uint8_t accel_flag;
uint8_t accel_data[6] = {0, 0, 0, 0, 0, 0};

int main(void)
{
    spi_init();
    LSM_init();
    usartd0_init();

    sei();

    while(1)
    {
        if (accel_flag == 1){

            // 1. READ DATA
            accel_data[0] = LSM_read(OUTX_L_XL);
            accel_data[1] = LSM_read(OUTX_H_XL);

            accel_data[2] = LSM_read(OUTY_L_XL);
            accel_data[3] = LSM_read(OUTY_H_XL);

            accel_data[4] = LSM_read(OUTZ_L_XL);
            accel_data[5] = LSM_read(OUTZ_H_XL);

            // 2. LITTLE ENDIAN USART

            usartd0_out_repeat(accel_data, 6);

            // 3. RESET AND ENABLE FLAG
            accel_flag = 0;
            PORTC.INT0MASK |= PIN6_bm;
        }
    }
}
```

```
// SUBROUTINES

void spi_init(void)
{
    /* Initialize the relevant SPI output signals to be in an "idle" state.
     * Refer to the relevant timing diagram within the LSM6DSL datasheet.
     * (You may wish to utilize the macros defined in `spi.h`.) */
    PORTF.OUTSET = SS_bm | SCK_bm;

    /* Configure the pin direction of relevant SPI signals. */
    PORTF.DIRSET = SS_bm | MOSI_bm | SCK_bm;
    PORTF.DIRCLR = MISO_bm;

    /* Set the other relevant SPI configurations. */
    SPIF.CTRL = SPI_PRESCALER_DIV16_gc |
                SPI_MASTER_bm |
                SPI_MODE_3_gc |
                SPI_ENABLE_bm;
    // This leaves CTRL.DORD false, so MSB -> LSB
}

void LSM_init(void)
{
    LSM_write(CTRL3_C, 0x01); // SW_RESET FLAG, SELF CLEARS
    for(uint16_t i = 0; i < 2000; i++);

    LSM_write(CTRL3_C, (1<<5)); // SET INTERRUPT TO ACTIVE-LOW
    LSM_write(INT1_CTRL, 0x01); // ENABLE INT1_DRDY_XL

    LSM_write(CTRL9_XL, 0); // DISABLE DEN IN DATA OUTPUT FOR HIGHER RES
    LSM_write(CTRL1_XL, 0b01010000); // 208Hz, +-2g
    // THIS MEANS 208 SAMPLES / SECOND

    PORTC.DIRCLR = PIN6_bm; // INT1 PIN
    PORTC.PIN6CTRL = PORT_ISC_LEVEL_gc; // LOW LEVEL INTERRUPT SENSE
    PORTC.INTCTRL = PORT_INT0LVL_LO_gc; // SET INT0 TO LO LVL INT
    PORTC.INT0MASK = PIN6_bm; // CONNECT PIN6 TO PORTC INT0
    PMIC.CTRL = PMIC_LOLVLEN_bm; // ENABLE LOLVL INT
}

void usartd0_out_repeat(uint8_t *data, uint8_t length)
{
    for (uint8_t i = 0; i < length; i++)
    {
        usartd0_out_char(data[i]);
    }
}

// INTERRUPTS

ISR(PORTC_INT0_vect)
{
    accel_flag = 1; // TELL USART DATA IS READY FOR TRANSMIT.

    PORTC.INT0MASK &= ~PIN6_bm; // DISABLE INTERRUPT

    PORTC.INTFLAGS = 0x01; // RESET INTERRUPT FLAG
}
```

}

Lab6_6 (EC)

```
/*
 * lab6_EC.c
 *
 * Created: 7/14/2026 5:48:49 PM
 * Author: Evan Baesler
 */

#include <avr/io.h>
#include <avr/interrupt.h>
#include "spi.h"
#include "usart.h"
#include "lsm6dsl.h"
#include "lsm6dsl_registers.h"

void spi_init(void);
void LSM_init(void);
void usartd0_out_repeat(uint8_t *data, uint8_t length);

volatile uint8_t gyro_flag;
int8_t gyro_data[6] = {0, 0, 0, 0, 0, 0};

int main(void)
{
    spi_init();
    LSM_init();
    usartd0_init();

    sei();

    while(1)
    {
        if (gyro_flag == 1)
        {
            // 1. READ DATA
            gyro_data[0] = LSM_read(OUTX_L_G);
            gyro_data[1] = LSM_read(OUTX_H_G);

            gyro_data[2] = LSM_read(OUTY_L_G);
            gyro_data[3] = LSM_read(OUTY_H_G);

            gyro_data[4] = LSM_read(OUTZ_L_G);
            gyro_data[5] = LSM_read(OUTZ_H_G);

            // 2. LITTLE ENDIAN USART
            usartd0_out_repeat(gyro_data, 6);

            // 3. RESET AND ENABLE FLAG
            gyro_flag = 0;
            PORTC.INT0MASK |= PIN6_bm;
        }
    }
}

// SUBROUTINES

void spi_init(void)
```

```
{

/* Initialize the relevant SPI output signals to be in an "idle" state.
 * Refer to the relevant timing diagram within the LSM6DSL datasheet.
 * (You may wish to utilize the macros defined in `spi.h`.) */
PORTF.OUTSET = SS_bm | SCK_bm;

/* Configure the pin direction of relevant SPI signals. */
PORTF.DIRSET = SS_bm | MOSI_bm | SCK_bm;
PORTF.DIRCLR = MISO_bm;

/* Set the other relevant SPI configurations. */
SPIF.CTRL = SPI_PRESCALER_DIV16_gc |
             SPI_MASTER_bm |
             SPI_MODE_3_gc |
             SPI_ENABLE_bm;
             // This leaves CTRL.DORD false, so MSB -> LSB
}

void LSM_init(void)
{
    LSM_write(CTRL3_C, 0x01); // SW_RESET FLAG, SELF CLEARS
    for(uint16_t i = 0; i < 2000; i++);

    LSM_write(CTRL3_C, (1<<5)); // SET INTERRUPT TO ACTIVE-LOW
    LSM_write(INT1_CTRL, (1<<1)); // ENABLE INT1_DRDY_G

    LSM_write(CTRL9_XL, 0); // DISABLE DEN IN DATA OUTPUT FOR HIGHER RES
    LSM_write(CTRL2_G, 0b01010010); // 208Hz, 125DPS
    // THIS MEANS 208 SAMPLES / SECOND

    PORTC.DIRCLR = PIN6_bm; // INT1 PIN
    PORTC.PIN6CTRL = PORT_ISC_LEVEL_gc; // LOW LEVEL INTERRUPT SENSE
    PORTC.INTCTRL = PORT_INT0LVL_LO_gc; // SET INT0 TO LO LVL INT
    PORTC.INT0MASK = PIN6_bm; // CONNECT PIN6 TO PORTC INT0
    PMIC.CTRL = PMIC_LOLVLEN_bm; // ENABLE LOLVL INT
}

void usartd0_out_repeat(uint8_t *data, uint8_t length)
{
    for (uint8_t i = 0; i < length; i++)
    {
        usartd0_out_char(data[i]);
    }
}

// INTERRUPTS

ISR(PORTC_INT0_vect)
{
    PORTC.INT0MASK &= ~PIN6_bm; // DISABLE INTERRUPT
    PORTC.INTFLAGS = 0x01; // RESET INTERRUPT FLAG

    gyro_flag = 1; // TELL USART DATA IS READY FOR TRANSMIT.
}
}
```

APPENDIX



Figure 1: SPI Waveforms Capture

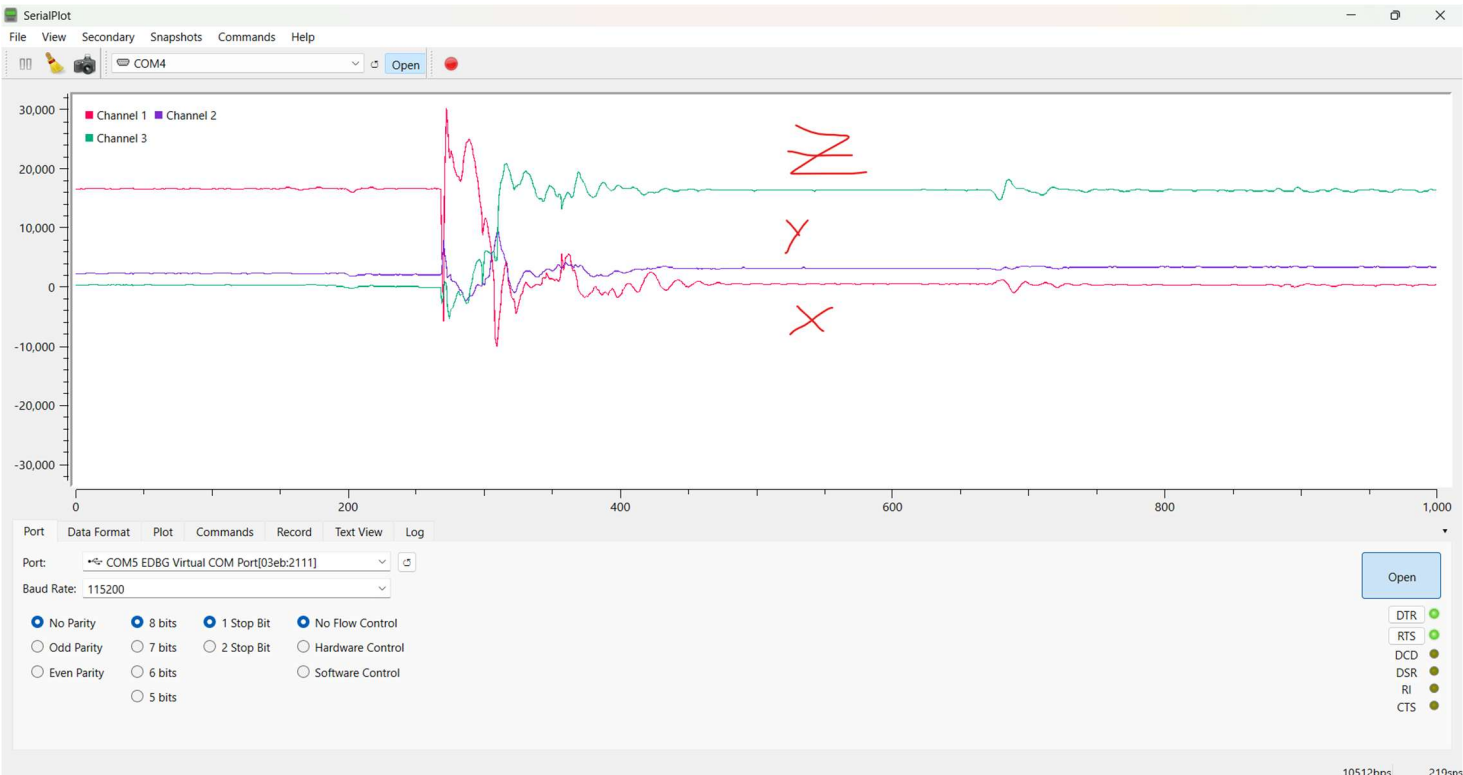


Figure 2: Accelerometer Reading

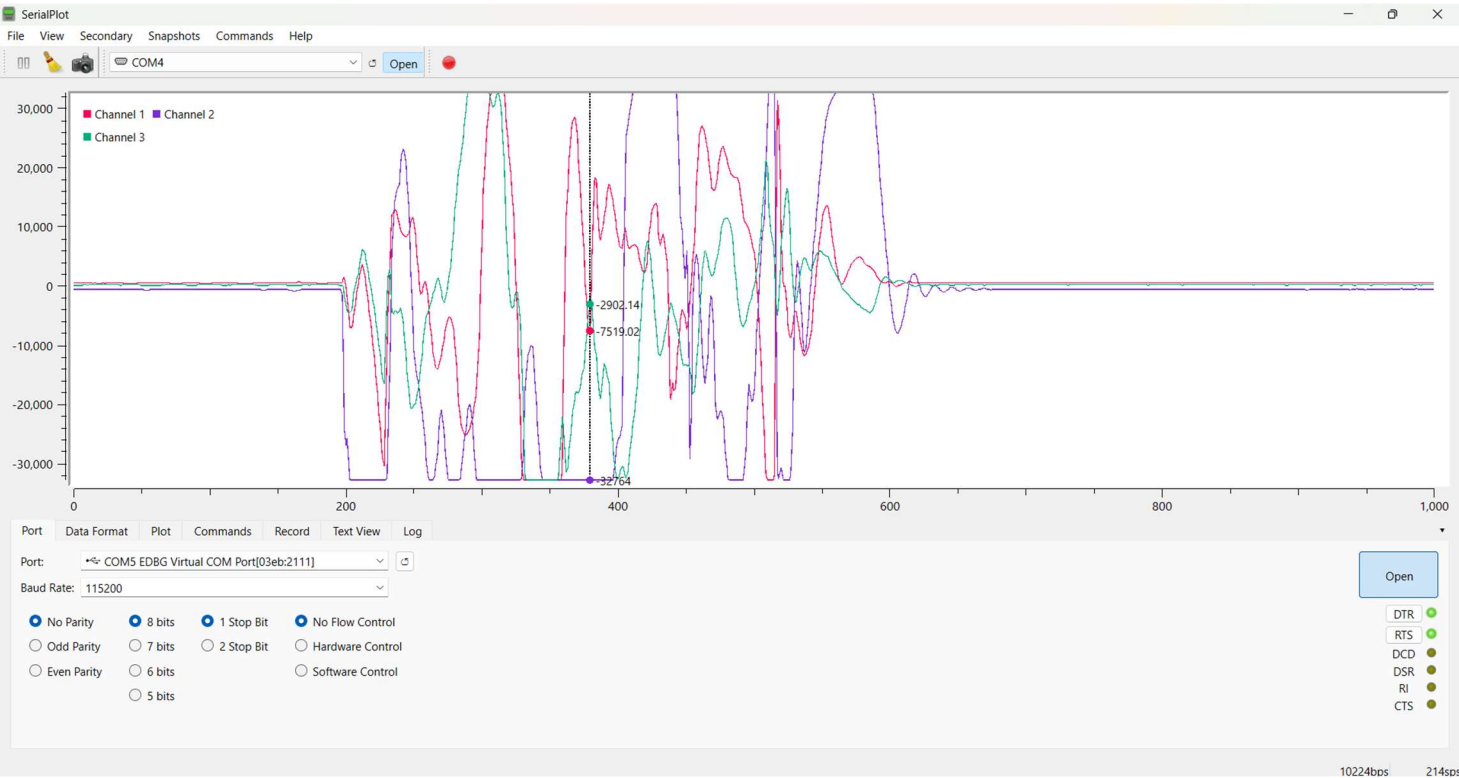


Figure 3: Gyroscope Reading

Word Address			
ATxmega128A1U		ATxmega64A1U	
0		0	Application Section (bytes) (128K/64K)
			...
FFFF	/	77FF	
F000	/	7800	Application Table Section (bytes) (8K/ 4K)
FFFF	/	7FFF	
10000	/	8000	Boot Section (bytes) (8K/4K)
10FFF	/	87FF	

Figure 4: Program Memory Addressing

Byte Address	ATxmega64A1U	Byte Address	ATxmega128A1U
0	I/O Registers (4K)	0	I/O Registers (4KB)
FFF		FFF	
1000	EEPROM (2K)	1000	EEPROM (2K)
17FF		17FF	
	RESERVED		RESERVED
2000	Internal SRAM (4K)	2000	Internal SRAM (8K)
2FFF		3FFF	
4000	External Memory (0 to 16MB)	3000	External Memory (0 to 16MB)
FFFFFF		FFFFFF	

Figure 5: Data Memory Addressing

13.16 Register summary – Ports

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Page
+0x00	DIR	DIR[7:0]								148
+0x01	DIRSET	DIRSET[7:0]								148
+0x02	DIRCLR	DIRCLR[7:0]								148
+0x03	DIRTGL	DIRTGL[7:0]								148
+0x04	OUT	OUT[7:0]								149
+0x05	OUTSET	OUTSET[7:0]								149
+0x06	OUTCLR	OUTCLR[7:0]								149
+0x07	OUTTGL	OUTTGL[7:0]								149
+0x08	IN	IN[7:0]								150
+0x09	INTCTRL	–	–	–	–	INT1LVL[1:0]		INT0LVL[1:0]		150
+0x0A	INT0MASK	INT0MSK[7:0]								150
+0x0B	INT1MASK	INT1MSK[7:0]								150
+0x0C	INTFLAGS	–	–	–	–	–	–	INT1IF	INT0IF	151
+0x0D	Reserved	–	–	–	–	–	–	–	–	
+0x0E	REMAP	–	–	SPI	USART0	TC0D	TC0C	TC0B	TC0A	151
+0x0F	Reserved	–	–	–	–	–	–	–	–	
+0x10	PIN0CTRL	SRLEN	INVEN	OPC[2:0]			ISC[2:0]			152
+0x11	PIN1CTRL	SRLEN	INVEN	OPC[2:0]			ISC[2:0]			152
+0x12	PIN2CTRL	SRLEN	INVEN	OPC[2:0]			ISC[2:0]			152
+0x13	PIN3CTRL	SRLEN	INVEN	OPC[2:0]			ISC[2:0]			152
+0x14	PIN4CTRL	SRLEN	INVEN	OPC[2:0]			ISC[2:0]			152
+0x15	PIN5CTRL	SRLEN	INVEN	OPC[2:0]			ISC[2:0]			152
+0x16	PIN6CTRL	SRLEN	INVEN	OPC[2:0]			ISC[2:0]			152
+0x17	PIN7CTRL	SRLEN	INVEN	OPC[2:0]			ISC[2:0]			152
+0x18	Reserved	–	–	–	–	–	–	–	–	
+0x19	Reserved	–	–	–	–	–	–	–	–	
+0x1A	Reserved	–	–	–	–	–	–	–	–	
+0x1B	Reserved	–	–	–	–	–	–	–	–	
+0x1C	Reserved	–	–	–	–	–	–	–	–	
+0x1D	Reserved	–	–	–	–	–	–	–	–	
+0x1E	Reserved	–	–	–	–	–	–	–	–	
+0x1F	Reserved	–	–	–	–	–	–	–	–	

Figure 6: Ports Register Summary

14.13 Register summary

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Page
+0x00	CTRLA	–	–	–	–	CLKSEL[3:0]				175
+0x01	CTRLB	CCDEN	CCCEN	CCBEN	CCAEN	–	WGMode[2:0]			175
+0x02	CTRLC	–	–	–	–	CMPD	CMPC	CMPB	CMPA	176
+0x03	CTRLD	EVACT[2:0]			EVDLY	EVSEL[3:0]				176
+0x04	CTRLF	–	–	–	–	–	–	BYTEM		178
+0x05	Reserved	–	–	–	–	–	–	–	–	
+0x06	INTCTRLA	–	–	–	–	ERRINTLVL[1:0]		OVINTLVL[1:0]		178
+0x07	INTCTRLB	CCCINTLVL[1:0]		CCBINTLVL[1:0]		CCAINTLVL[1:0]		CCINTLVL[1:0]		178
+0x08	CTRLCLR	–	–	–	QDECINDX	CMD[1:0]		LUPD	DIR	179
+0x09	CTRLFSET	–	–	–	QDECINDX	CMD[1:0]		LUPD	DIR	180
+0x0A	CTRLGCLR	–	–	–	CCDBV	CCCBV	CCBBV	CCABV	PERBV	180
+0x0B	CTRLGSET	–	–	–	CCDBV	CCCBV	CCBBV	CCABV	PERBV	180
+0x0C	INTFLAGS	CCDIF	CCCIF	CCBIF	CCAIF	–	–	ERRIF	OVFIF	180
+0x0D	Reserved	–	–	–	–	–	–	–	–	
+0x0E	Reserved	–	–	–	–	–	–	–	–	
+0x0F	TEMP	TEMP[7:0]								181
+0x10 to +0x1F	Reserved	–	–	–	–	–	–	–	–	
+0x20	CNTL	CNT[7:0]								181
+0x21	CNTH	CNT[15:8]								181
+0x22 to +0x25	Reserved	–	–	–	–	–	–	–	–	
+0x26	PERL	PER[7:0]								182
+0x27	PERH	PER[8:15]								182
+0x28	CCAL	CCA[7:0]								182
+0x29	CAHA	CCA[15:8]								182
+0x2A	CCBL	CCB[7:0]								182
+0x2B	CCBH	CCB[15:8]								182
+0x2C	CCCL	CCC[7:0]								182
+0x2D	CCCH	CCC[15:8]								182
+0x2E	CCDL	CCD[7:0]								182
+0x2F	CCDH	CCD[15:8]								182
+0x30 to +0x35	Reserved	–	–	–	–	–	–	–	–	
+0x36	PERBUFL	PERBUF[7:0]								183

Figure 7: Timer/Counter Register Summary

12.9 Register summary

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Page
+0x00	STATUS	NMIEX	–	–	–	–	HILVLEX	MEDLVLEX	LOLVLEX	136
+0x01	INTPRI	INTPRI[7:0]								136
+0x02	CTRL	RREN	IVSEL	–	–	–	HILVLEN	MEDLVLEN	LOLVLEN	137

Figure 8: Interrupts Register Summary

27.12 Register summary – EBI

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Page	
+0x00	CTRL	SDDATAW[1:0]		LPCMODE[1:0]		SRMODE[1:0]		IFMODE[1:0]		329	
+0x01	SDRAMCTRLA	—	—	—	—	SDCAS	SDROW	SDCOL[1:0]		330	
+0x02	Reserved	—	—	—	—	—	—	—	—		
+0x03	Reserved	—	—	—	—	—	—	—	—		
+0x04	REFRESHL	SDRAM Refresh Period Low Byte								331	
+0x05	REFRESHH	—	—	—	—	—	—	SDRAM Refresh Period High		331	
+0x06	INITDLYL	SDRAM Initialization Time Low Byte								331	
+0x07	INITDLYH	—	—	SDRAM Initialization Time High Byte							331
+0x08	SDRAMCTRLB	MRDLY[1:0]		ROWCYCDLY[[2:0]			RPDLY[2:0]			332	
+0x09	SDRAMCTRLC	WRDLY[1:0]		ESRDLY[2:0]			ROWCOLDLY[2:0]			333	
+0x0A	Reserved	—	—	—	—	—	—	—	—		
+0x0B	Reserved	—	—	—	—	—	—	—	—		
+0x0C	Reserved	—	—	—	—	—	—	—	—		
+0x0D	Reserved	—	—	—	—	—	—	—	—		
+0x0E	Reserved	—	—	—	—	—	—	—	—		
+0x0F	Reserved	—	—	—	—	—	—	—	—		
+0x10	CS0	Chip Select 0 Offset Address									
+0x14	CS1	Chip Select 1 Offset Address									
+0x18	CS2	Chip Select 2 Offset Address									
+0x1C	CS3	Chip Select 3 Offset Address									

Figure 9: EBI Register Summary

27.13 Register summary – EBI chip select

Address	Name		Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Page
+0x00	CTRLA		–	ASIZE[4:0]					MODE[1:0]		329
+0x01	CTRLB	(SRAM)	–	–	–	–	–	SRWS[2:0]			330
		(SDRAM)	SDINITDONE	–	–	–	–	SDSREN	SDMODE[1:0]		
+0x02	BASEADDRL		Chip Select Base Address Low Byte				–	–	–	–	337
+0x03	BASEADDRH		Chip Select Base Address High Byte								337

Figure 10: EBI Chip Select Register Summary

Table 33-7. Port H - alternate functions.

PORT H	PIN #	INTERRUPT	SDRAM 3P	SRAM ALE1	SRAM ALE12	LPC3 ALE1	LPC2 ALE1	LPC2 ALE12
GND	53							
VCC	54							
PH0	55	SYNC	$\overline{WE}$	$\overline{WE}$	$\overline{WE}$	$\overline{WE}$	$\overline{WE}$	$\overline{WE}$
PH1	56	SYNC	$\overline{CAS}$	$\overline{RE}$	$\overline{RE}$	$\overline{RE}$	$\overline{RE}$	$\overline{RE}$
PH2	57	SYNC/ASYN	$\overline{RAS}$	$\overline{ALE1}$	$\overline{ALE1}$	$\overline{ALE1}$	$\overline{ALE1}$	$\overline{ALE1}$
PH3	58	SYNC	$\overline{DQM}$		$\overline{ALE2}$			$\overline{ALE2}$
PH4	59	SYNC	BA0	$\overline{CS0}/A16$	$\overline{CS0}$	$\overline{CS0}/A16$	$\overline{CS0}$	$\overline{CS0}/A16$
PH5	60	SYNC	BA1	$\overline{CS1}/A17$	$\overline{CS1}$	$\overline{CS1}/A17$	$\overline{CS1}$	$\overline{CS1}/A17$
PH6	61	SYNC	CKE	$\overline{CS2}/A18$	$\overline{CS2}$	$\overline{CS2}/A18$	$\overline{CS2}$	$\overline{CS2}/A18$
PH7	62	SYNC	CLK	$\overline{CS3}/A19$	$\overline{CS3}$	$\overline{CS3}/A19$	$\overline{CS3}$	$\overline{CS3}/A19$

Notes: 1. CS0 - CS3 can optionally be moved to Port E or F
2. A16-A23 can optionally be moved to Port E or F when EBI configured in 4PORT mode. Refer to the EBIOUT register description in the XMEGA AU Manual.

Figure 11: PORTH Alternate Functions

Table 33-8. Port J - alternate functions.

PORT J	PIN #	INTERRUPT	SDRAM 3P	SRAM ALE1	SRAM ALE12	LPC3 ALE1	LPC2 ALE1	LPC2 ALE12
GND	63							
VCC	64							
PJ0	65	SYNC	D0	D0	D0	D0/A0	D0/A0	D0/A0/A8
PJ1	66	SYNC	D1	D1	D1	D1/A1	D1/A1	D1/A1/A9

ATxmega128A1U/ATxmega64A1U

PORT J	PIN #	INTERRUPT	SDRAM 3P	SRAM ALE1	SRAM ALE12	LPC3 ALE1	LPC2 ALE1	LPC2 ALE12
PJ2	67	SYNC/ASYN	D2	D2	D2	D2/A2	D2/A2	D2/A2/A10
PJ3	68	SYNC	D3	D3	D3	D3/A3	D3/A3	D3/A3/A11
PJ4	69	SYNC	A8	D4	D4	D4/A4	D4/A4	D4/A4/A12
PJ5	70	SYNC	A9	D5	D5	D5/A5	D5/A5	D5/A5/A13
PJ6	71	SYNC	A10	D6	D6	D6/A6	D6/A6	D6/A6/A14
PJ7	72	SYNC	A11	D7	D7	D7/A7	D7/A7	D7/A7/A15

Figure 12: PORTJ Alternate Functions

Table 33-9. Port K - alternate functions.

PORT K	PIN #	INTERRUPT	SDRAM 3P	SRAM ALE1	SRAM ALE2	LPC3 ALE1
GND	73					
VCC	74					
PK0	75	SYNC	A0	A0/A8	A0/A8/A16	A8
PK1	76	SYNC	A1	A1/A9	A1/A9/A17	A9
PK2	77	SYNC/ASYNC	A2	A2/A10	A2/A10/A18	A10
PK3	78	SYNC	A3	A3/A11	A3/A11/A19	A11
PK4	79	SYNC	A4	A4/A12	A4/A12/A20	A12
PK5	80	SYNC	A5	A5/A13	A5/A13/A21	A13
PK6	81	SYNC	A6	A6/A14	A6/A14/A22	A14
PK7	82	SYNC	A7	A7/A15	A7/A15/A23	A15

Figure 13: PORTK Alternate Functions

23.16 Register summary

23.16.1 Register description – USART

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Page
+0x00	DATA	DATA[7:0]								294
+0x01	STATUS	RXCIF	TXCIF	DREIF	FERR	BUOVF	PERR	–	RXB8	294
+0x02	Reserved	–	–	–	–	–	–	–	–	
+0x03	CTRLA	–	–	RXCINTLVL[1:0]		TXCINTLVL[1:0]		DREINTLVL[1:0]		295
+0x04	CTRLB	–	–	–	RXEN	TXEN	CLK2X	MPCM	TXB8	296
+0x05	CTRLC	CMODE[1:0]		PMODE[1:0]		SBMODE	CHSIZE[2:0]			296
+0x06	BAUDCTRLA	BSEL[7:0]								298
+0x07	BAUDCTRLB	BSCALE[3:0]				BSEL[11:8]				298

23.16.2 Register description – USART in SPI Master Mode

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Page
+0x00	DATA	DATA[7:0]								294
+0x01	STATUS	RXCIF	TXCIF	DREIF	–	–	–	–	–	294
+0x02	Reserved	–	–	–	–	–	–	–	–	
+0x03	CTRLA	–	–	RXCINTLVL[1:0]		TXCINTLVL[1:0]		DREINTLVL[1:0]		295
+0x04	CTRLB	–	–	–	RXEN	TXEN	–	–	–	296
+0x05	CTRLC	CMODE[1:0]		–	–	–	UDORD	UCPHA	–	296
+0x06	BAUDCTRLA	BSEL[7:0]								298
+0x07	BAUDCTRLB	BSCALE[3:0]				BSEL[11:8]				298

Figure 14: USART Register Summaries

Table 33-3. Port C - alternate functions.

PORT C	PIN#	INTERRUPT	TCC0 ⁽¹⁾⁽²⁾	AWEXC	TCC1	USART0 ⁽³⁾	USART1	SPIC ⁽⁴⁾	TWIC	CLOCKOUT ⁽⁵⁾	EVENTOUT ⁽⁶⁾
GND	13										
VCC	14										
PC0	15	SYNC	OC0A	OC0ALS					SDA		

ATxmega128A1U/ATxmega64A1U

PORT C	PIN#	INTERRUPT	TCC0 ⁽¹⁾⁽²⁾	AWEXC	TCC1	USART0 ⁽³⁾	USART1	SPIC ⁽⁴⁾	TWIC	CLOCKOUT ⁽⁵⁾	EVENTOUT ⁽⁶⁾
PC1	16	SYNC	OC0B	OC0AHS		XCK0			SCL		
PC2	17	SYNC/ASYNC	OC0C	OC0BLS		RXD0					
PC3	18	SYNC	OC0D	OC0BHS		TXD0					
PC4	19	SYNC		OC0CLS	OC1A			SS			
PC5	20	SYNC		OC0CHS	OC1B		XCK1	MOSI			
PC6	21	SYNC		OC0DLS			RXD1	MISO		clk _{RTC}	
PC7	22	SYNC		OC0DHS			TXD1	SCK		clk _{PER}	EVOUT

- Notes:
1. Pin mapping of all TC0 can optionally be moved to high nibble of port. Refer to Pin Remap register in I/O Ports in the XMEGA AU Manual.
 2. If TC0 is configured as TC2 all eight pins can be used for PWM output. Refer to Pin Remap register in I/O Ports in the XMEGA AU Manual..
 3. Pin mapping of all USART0 can optionally be moved to high nibble of port. Refer to Pin Remap register in I/O Ports in the XMEGA AU Manual..
 4. Pins MOSI and SCK for all SPI can optionally be swapped. Refer to Pin Remap register in I/O Ports in the XMEGA AU Manual.
 5. CLKOUT can optionally be moved between port C, D and E and between pin 4 and 7. Refer to CLKEVOUT register in I/O port configuration in the XMEGA AU Manual.
 6. EVOUT can optionally be moved between port C, D and E and between pin 4 and 7. Refer to CLKEVOUT register in I/O port configuration in the XMEGA AU Manual.

Figure 15: PORTC Alternate Functions

Notes: 1. All pins on the port can optionally be used for EBI chip select or address lines. Refer to the EBIOUT register description in the XMEGA AU Manual.

Table 33-6. Port F - alternate functions.

PORT F	PIN #	INTERRUPT	TCF0	TCF1	USARTF0	USARTF1	SPIF	TWIF
GND	43							
VCC	44							
PF0	45	SYNC	OC0A					SDA
PF1	46	SYNC	OC0B		XCK0			SCL
PF2	47	SYNC/ASYNC	OC0C		RXD0			
PF3	48	SYNC	OC0D		TXD0			
PF4	49	SYNC		OC1A			$\overline{SS}$	
PF5	50	SYNC		OC1B		XCK1	MOSI	
PF6	51	SYNC				RXD1	MISO	
PF7	52	SYNC				TXD1	SCK	

Note: 1. All pins on the port can optionally be used for EBI chip select or address lines. Refer to the EBIOUT register description in the XMEGA AU Manual.

Figure 16: PORTF Alternate Functions

22.8 Register summary

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Page
+0x00	CTRL	CLK2X	ENABLE	DORD	MASTER	MODE[1:0]		PRESCALER[1:0]		276
+0x01	INTCTRL	–	–	–	–	–	–	INTLVL[1:0]		277
+0x02	STATUS	IF	WRCOL	–	–	–	–	–	–	277
+0x03	DATA	DATA[7:0]								278

22.9 Interrupt vector summary

Table 22-4. SPI interrupt vector and its offset word address.

Offset	Source	Interrupt Description
0x00	SPI_vect	SPI interrupt vector

Figure 17: SPI Register Summary

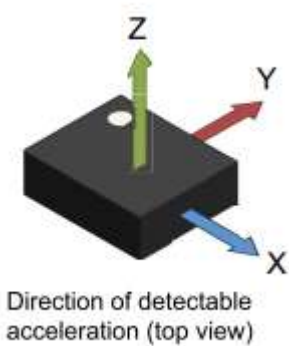


Figure 18: Axis Directions for IMU